Instituto Tecnológico de Mazatlán

Ingeniería en Sistemas Computacionales

PRÁCTICA 2

Conexión remota de una aplicación C# a una base de datos MySQL

Informe

INTEGRANTES

Sánchez Moreno César Adrián

Piña Ríos Fernando Emmanuel

Mazatlán, Sinaloa · 16 de octubre de 2026

1. Introducción

En esta práctica se modificó una aplicación de escritorio desarrollada en C# con SharpDevelop, una agenda estudiantil con fotografía, para que dejara de depender de una base de datos local y pudiera trabajar contra un servidor MySQL ubicado en otra computadora de la misma red. La base de datos, llamada *agenda*, contiene las tablas *datos_personales* y *direccion* y se administra con phpMyAdmin.

El escenario de trabajo usa dos laptops: una con Ubuntu, que aloja MySQL, y otra con Windows, donde se ejecuta el programa. Ambas se conectan a la misma red, ya sea una red Wi-Fi o el hotspot de un celular. Como la dirección IP del servidor cambia al cambiar de red, el programa debía permitir modificarla sin recompilar.

2. Objetivos

Objetivo general. Conectar el programa de la agenda, ejecutado en Windows, con la base de datos MySQL alojada en una laptop con Ubuntu, a través de una red local.

Objetivos específicos.

- Configurar MySQL en Ubuntu para aceptar conexiones remotas de forma controlada.
- Permitir el acceso por el puerto 3306 a través del firewall.
- Adaptar el programa para que la IP del servidor se capture en pantalla y no esté escrita en el código.
- Resolver los errores de compatibilidad entre el conector de .NET y MySQL 8.4.
- Verificar las operaciones de consulta, alta, edición y baja de registros desde el equipo remoto.

3. Entorno de trabajo

Elemento	Servidor	Cliente
Sistema operativo	Ubuntu (laptop)	Windows (laptop)
Software principal	MySQL 8.4.11, Apache y phpMyAdmin	SharpDevelop 4.4, C# Windows Forms
Librería de acceso	No aplica	MySql.Data (Connector/NET), .NET Framework 4.0
Función	Aloja la base de datos *agenda*	Ejecuta la agenda estudiantil
Dirección IP (red de prueba)	192.168.1.82	192.168.1.88

Las direcciones IP corresponden a la red doméstica en la que se hicieron las primeras pruebas. En otra red o con otro hotspot cambian y deben consultarse de nuevo.

4. Desarrollo

4.1 Cuenta de usuario con acceso remoto

En MySQL una cuenta se identifica por usuario y host. La cuenta original, *'fernando-pina'@'localhost'*, solo aceptaba conexiones desde el mismo equipo. Para permitir conexiones desde otra laptop se renombró la cuenta para que acepte cualquier host, conservando su contraseña y sus privilegios:

```
RENAME USER 'fernando-pina'@'localhost' TO 'fernando-pina'@'%';
```

4.2 Escucha en la red y firewall

MySQL se configuró para escuchar en todas las interfaces de red mediante *bind-address = 0.0.0.0* en el archivo */etc/mysql/mysql.conf.d/mysqld.cnf*. Después se abrió el puerto de MySQL en el firewall de Ubuntu:

```
sudo ufw allow 3306/tcp
```

Se comprobó con el comando *ss* que el servicio quedara escuchando en *0.0.0.0:3306*.

4.3 Compatibilidad de autenticación con MySQL 8.4

MySQL 8.4 desactiva por defecto el plugin *mysql_native_password* y usa *caching_sha2_password*. El conector MySql.Data que utiliza el proyecto es antiguo y no soporta ese método, por lo que fue necesario habilitar el plugin clásico, indicarlo como método predeterminado del servidor y cambiar el método de la cuenta. En *mysqld.cnf*, bajo *[mysqld]*:

```
mysql_native_password=ON
authentication_policy=mysql_native_password
```

Luego se reinició el servicio y se actualizó la cuenta:

```
sudo systemctl restart mysql

ALTER USER 'fernando-pina'@'%'
  IDENTIFIED WITH mysql_native_password BY 'TU_CONTRASEÑA';
FLUSH PRIVILEGES;
```

4.4 Cambios en el programa

Se reorganizó el código de *MainForm.cs* y se agregó al formulario un cuadro de texto, *Servidor (IP)*, para escribir la dirección del servidor. Los cambios principales fueron:

- La cadena de conexión se construye en el momento de pulsar **Conectar**, con la IP capturada, así que ya no se recompila al cambiar de red.
- La última IP usada se guarda en un archivo *servidor.txt* junto al ejecutable y se recupera al abrir el programa.

- Se añadió *SslMode=None* a la cadena de conexión para evitar el fallo de negociación SSL del conector antiguo.
- Se mostraron mensajes de error claros para servidor inalcanzable, credenciales incorrectas y base de datos inexistente.
- Las operaciones de alta, edición y baja usan consultas con parámetros y transacciones. Con esto se eliminó la concatenación de texto en el borrado, que era vulnerable a inyección SQL.

Cadena de conexión resultante:

```
string.Format("Server={0};Port={1};Database={2};Uid={3};Pwd={4};"
  + "Connection Timeout=8;SslMode=None;",
  servidor, PUERTO, BASE_DATOS, USUARIO, CONTRASENA);
```

5. Conclusiones

Los resultados de las pruebas y las incidencias encontradas se presentan en el documento *Resultados de la práctica 2*, que acompaña a este informe.

La práctica mostró que conectar una aplicación a una base de datos remota requiere configurar varias capas a la vez: la red y el firewall, la cuenta de usuario, el método de autenticación y la propia aplicación. La mayor dificultad fue la incompatibilidad entre un conector antiguo de .NET y las medidas de seguridad predeterminadas de MySQL 8.4. Se resolvió identificando cada mensaje de error y aplicando el ajuste correspondiente.

Parametrizar la IP en la interfaz hizo al programa portátil entre redes, algo necesario porque la dirección del servidor cambia con cada red. Como mejora futura se recomienda actualizar el conector MySql.Data y migrar el proyecto a una versión más reciente de .NET. Así podría activarse de nuevo SSL y volver al método de autenticación predeterminado de MySQL, que es más seguro. También conviene limitar el acceso remoto de la cuenta a la IP del equipo cliente en lugar de usar el comodín %.